CSA1709 - Artificial Intelligence - SLOT: B

Lab Experiments

Experiment 13: Minimax Algorithm for Gaming

Aim : To implement the Minimax algorithm in Python for a two-player game and determine the best possible move for a player by considering all possible game states.

Algorithm :

1. Create a game state or game tree.
2. Define terminal states with their utility values.
3. The Maximizer tries to obtain the maximum score.
4. The Minimizer tries to obtain the minimum score.
5. Recursively evaluate all possible moves.
6. Select the maximum value for the Maximizer.
7. Select the minimum value for the Minimizer.
8. Continue until the optimal move is obtained.
9. Display the best move and its score.

Program :

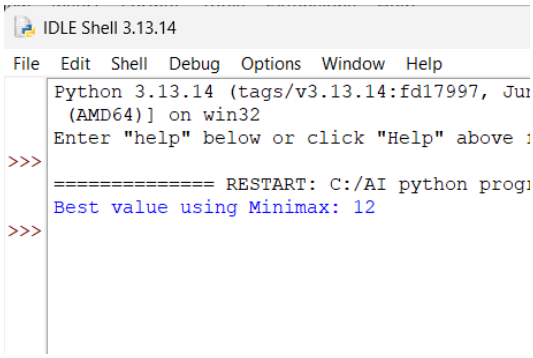
```
def minimax(depth, node, maximizing_player, tree):
    if depth == 3:
        return tree[node]
    if maximizing_player:
        best = -9999
        for child in [2 * node, 2 * node + 1]:
            value = minimax(depth + 1, child, False, tree)
            best = max(best, value)
        return best
    else:
        best = 9999
        for child in [2 * node, 2 * node + 1]:
            value = minimax(depth + 1, child, True, tree)
            best = min(best, value)
```

```
        return best
tree = {
    8: 3,
    9: 5,
    10: 2,
    11: 9,
    12: 12,
    13: 5,
    14: 23,
    15: 23
}
best_value = minimax(0, 1, True, tree)
print("Best value using Minimax:", best_value)
```

Sample Input:

Game Tree Leaf Values:
3 5 2 9 12 5 23 23

Output :



Result:

Thus, the Minimax algorithm was successfully implemented in Python to determine the best possible outcome for the maximizing player in a two-player game.

Experiment 14. Alpha-Beta Pruning Algorithm for Gaming

Aim: To implement the Alpha-Beta Pruning algorithm in Python to optimize the Minimax search by eliminating branches that cannot affect the final decision.

Algorithm:

1. Start with the root node of the game tree.
2. Initialize Alpha = -∞ and Beta = +∞.
3. Alpha represents the best value found for the Maximizer.
4. Beta represents the best value found for the Minimizer.
5. Explore the game tree recursively.
6. For Maximizer, update Alpha with the maximum value.
7. For Minimizer, update Beta with the minimum value.
8. If Beta ≤ Alpha, stop exploring the remaining branches.
9. Return the optimal value.
10. Display the best value obtained.

Program:

```
def alpha_beta(depth, node, maximizing_player,
               alpha, beta, tree):
    if depth == 3:
        return tree[node]
    if maximizing_player:
        best = -9999
        for child in [2 * node, 2 * node + 1]:
            value = alpha_beta(
                depth + 1,
                child,
                False,
```

```
            alpha,
            beta,
            tree
        )
        best = max(best, value)
        alpha = max(alpha, best)
        if beta <= alpha:
            break
    return best
else:
    best = 9999
    for child in [2 * node, 2 * node + 1]:
        value = alpha_beta(
            depth + 1,
            child,
            True,
            alpha,
            beta,
            tree
        )
        best = min(best, value)
        beta = min(beta, best)
```

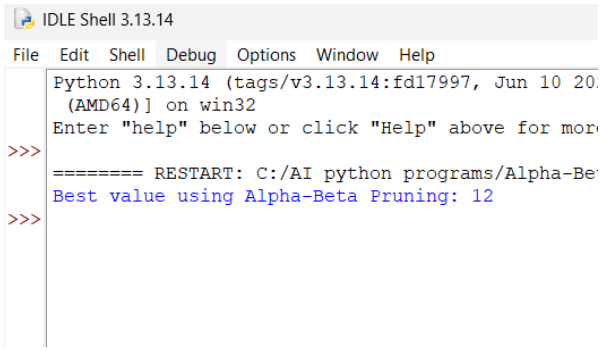
```
            if beta <= alpha:
                break
        return best
tree = {
    8: 3,
    9: 5,
    10: 2,
    11: 9,
    12: 12,
    13: 5,
    14: 23,
    15: 23
}
alpha = -9999
beta = 9999
best_value = alpha_beta(
    0, 1, True,
    alpha, beta, tree
)
print("Best value using Alpha-Beta Pruning:", best_value)
```

Sample Input:

Game Tree Leaf Values:

3 5 2 9 12 5 23 23

Output:



Result:

Thus, the Alpha-Beta Pruning algorithm was successfully implemented in Python. It obtained the optimal value while reducing unnecessary exploration of the game tree.

Experiment 15. Decision Tree

Aim: To implement a Decision Tree classification algorithm in Python and classify data based on its features.

Algorithm:

1. Import the required libraries.
2. Create a dataset containing input features and target classes.
3. Separate the features and target values.
4. Create a Decision Tree Classifier.
5. Train the classifier using the dataset.
6. Provide a new input for prediction.
7. Predict the class of the new input.
8. Display the predicted result.

Program:

```
data = [
    ["Sunny", "Hot", "Yes"],
    ["Sunny", "Cold", "Yes"],
    ["Rainy", "Hot", "No"],
    ["Rainy", "Cold", "No"],
    ["Sunny", "Hot", "Yes"],
    ["Rainy", "Cold", "No"]
]
def decision_tree(weather, temperature):
    if weather == "Sunny":
        return "Yes"
    elif weather == "Rainy":
        return "No"
    else:
        return "Unknown"
print("Decision Tree Classification")
weather = input("Enter Weather (Sunny/Rainy): ")
temperature = input("Enter Temperature (Hot/Cold): ")
result = decision_tree(weather, temperature)
print("Weather:", weather)
print("Temperature:", temperature)
```

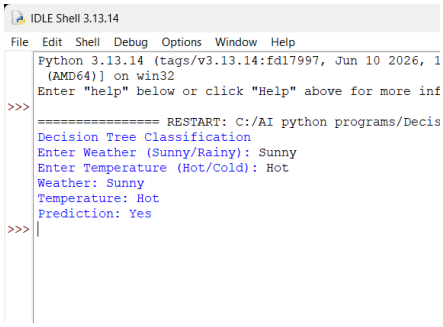
```
print("Prediction:", result)
```

Sample Input :

Enter Weather (Rainy, Sunny): Sunny

Enter Temperature (Cold, Hot): Hot

Output:



Result :

Thus, the Decision Tree classification algorithm was successfully implemented in Python and used to predict the class of a new data instance.

Experiment 16. Feed Forward Neural Network

Aim: To implement a Feed Forward Neural Network (FFNN) in Python for classification. The network passes input data forward through the hidden layer to the output layer without feedback connections.

Algorithm:

1. Import the required libraries.
2. Prepare the input dataset and target values.
3. Divide the dataset into training and testing data.

4. Create a feed-forward neural network.
5. Define the input, hidden, and output layers.
6. Train the neural network using the training data.
7. Test the trained model using test data.
8. Calculate the prediction accuracy.
9. Display the predicted and actual values.

Program:

```
import math

def sigmoid(x):

    return 1 / (1 + math.exp(-x))

x1 = float(input("Enter first input (0 or 1): "))

x2 = float(input("Enter second input (0 or 1): "))

w1 = 0.5

w2 = 0.5

b1 = 0.5

hidden_input = (x1 * w1) + (x2 * w2) + b1

hidden_output = sigmoid(hidden_input)

w3 = 1.0

b2 = -0.5

output_input = (hidden_output * w3) + b2

output = sigmoid(output_input)

print("\nFeed Forward Neural Network")

print("----------------------------")

print("Input 1       :", x1)
```

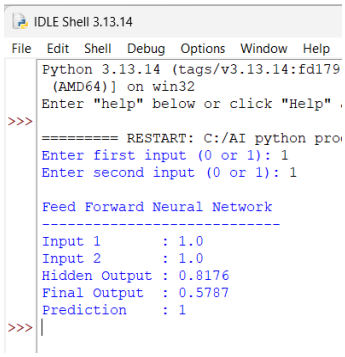
```
print("Input 2       :", x2)

print("Hidden Output :", round(hidden_output, 4))

print("Final Output  :", round(output, 4))

if output >= 0.5:

    print("Prediction    : 1")

else:

    print("Prediction    : 0")
```

Sample Input

Enter first input (0 or 1): 1

Enter second input (0 or 1): 1

Output :



Result:

Thus, the Feed Forward Neural Network was successfully implemented in Python and trained to classify the given input data.